SECP3133

High Performance Data Processing

Assignment 2

Mastering Big Data Handling

Student Briefing and Guidance Document

Due Date	Group Size	Platform	Dataset Size
4 May 2025, 5:00 PM	2 Students per Group	GitHub + Google Colab	Minimum 700 MB

1. Assignment Overview

1.1 Purpose of This Assignment

Welcome to Assignment 2 of SECP3133 **High Performance Data Processing**. This assignment is titled **Mastering Big Data Handling**, and it is designed to give you hands-on experience working with large, real-world datasets that go beyond the comfortable limits of everyday data processing tools. By completing this assignment, you will move from being a student who understands big data in theory to a practitioner who can actually manage it in practice.

In today's technology landscape, virtually every industry generates enormous volumes of data every second. From e-commerce transactions and social media interactions to sensor readings in manufacturing plants and electronic medical records in hospitals, the scale of data is unprecedented. Traditional tools and methods that work well on small datasets often fail, crash, or slow to a halt when confronted with hundreds of megabytes or gigabytes of information. This assignment challenges you to face that problem directly and solve it.

Specifically, you and your partner will select a dataset that is **larger than 700 megabytes**, and you will apply a range of big data handling strategies to load, process, and analyse it efficiently. You will compare how traditional approaches using the Pandas library perform against more scalable solutions, and you will document your findings clearly in both a Markdown report and an annotated Jupyter notebook.

1.2 Why Big Data Handling Matters

Understanding how to handle large datasets is not merely an academic exercise. It is one of the most practical and commercially valuable skills a data engineer, data scientist, or software developer can possess. When a data pipeline takes hours to run because of inefficient loading and processing, an organisation loses money, time, and competitive advantage. When a notebook crashes because an analyst tried to load a 2 GB CSV file into memory all at once, deadlines are missed and work must be redone.

The strategies you will learn and apply in this assignment such as chunking, data type optimisation, sampling, and parallel processing are exactly the techniques used by professional data engineers every day at companies like Google, Amazon, and financial institutions worldwide. Mastering these techniques here, in a structured academic setting, prepares you to step into those professional environments with confidence.

1.3 Connection to SECP3133 Concepts

This assignment directly connects to the core themes of this subject. Throughout SECP3133, we have been examining how to process data at scale, how to measure and improve computational performance, and how to choose the right tools for the right job. Assignment 2 is where all of those conceptual threads come together. You will see memory optimisation in action, you will measure execution time differences between approaches, and you will make informed decisions about which libraries best serve a given processing task.

Key Link to Subject Outcomes

Performance Analysis: You will measure and compare real execution times and memory usage.

Scalable Computing: You will use libraries specifically designed to handle large scale data.

Practical Implementation: You will write and test working Python code on a real dataset.

2. Learning Outcomes

By the end of this assignment, you are expected to demonstrate competence across three areas of learning: understanding, application, and comparison. Each of these is important, and your marks will reflect your depth of engagement with all three.

2.1 What You Will Understand

You will develop a clear understanding of why traditional data processing tools reach their limits with large datasets. You will be able to articulate the specific challenges such as excessive memory consumption, slow read times, and processing bottlenecks that arise when naive approaches are used on big data. You will also understand the trade-offs involved in different handling strategies, knowing that no single solution is perfect in every situation.

2.2 What You Will Apply

You will apply at least five big data handling strategies using Python code in a Google Colab notebook. These strategies include loading only the necessary portions of a dataset, processing data in chunks, optimising column data types to reduce memory footprint, applying sampling for fast prototyping, and using parallel processing through scalable libraries. You will also document your implementation clearly so that another person reading your notebook can understand both what your code does and why you wrote it that way.

2.3 What You Will Compare

You will conduct a structured comparative analysis that places your traditional Pandas baseline against your scalable library implementations. This comparison will be grounded in measurable evidence: **memory usage figures**, **execution time measurements**, and a discussion of **processing efficiency**. You will present your findings clearly, ideally using tables and charts, and you will reflect critically on what the data tells you about the strengths and limitations of each approach.

3. Core Technical Requirements

Before diving into the individual tasks, you need to understand the technical framework within which this assignment operates. These requirements are not optional additions. They are fundamental to how you will be assessed.

3.1 Library Requirements

Each group must use **exactly three Python libraries** for this assignment. The selection of these libraries is not entirely free. There is one compulsory library and you must choose the remaining two from a set of approved scalable options.

Library Selection Rules

Library 1 (Compulsory): Pandas must be used as your baseline library. Pandas is the most widely used data manipulation library in Python, and it will serve as your reference point for performance comparison. Every group must use Pandas, and there are no exceptions to this rule.

Library 2 and Library 3 (Scalable): You must choose two additional libraries from scalable options such as **Dask**, **PyArrow**, or **Polars**. These libraries are designed to handle large scale data more efficiently than standard Pandas. Each of these libraries has distinct strengths, and part of your learning is understanding what makes them different from one another.

3.2 Brief Description of Approved Scalable Libraries

Dask is a parallel computing library that mirrors the Pandas API but operates on data that is too large to fit into memory by breaking it into smaller partitions and processing them in parallel. It is particularly well suited to large CSV files and dataframe operations.

PyArrow is a cross-language development platform for in-memory analytics, based on the Apache Arrow format. It is extremely fast for reading and writing large Parquet and CSV files, and it excels at columnar data operations with very low memory overhead.

Polars is a relatively newer library written in Rust and designed from the ground up for performance. It supports lazy evaluation, which means it builds a query plan before executing it, allowing it to optimise operations automatically. It is known for exceptional speed on large datasets.

3.3 What Your Analysis Must Include

Your work must go beyond simply running code. The following components are required for all submissions, and the depth with which you address them will significantly affect your grade.

- A performance comparison between Pandas and each of your two scalable libraries, measuring memory usage and execution time.

- A memory optimisation analysis showing how different strategies and libraries affect the amount of RAM used during processing.
- For High Distinction level work, a deep performance analysis that goes beyond surface-level numbers. This means interpreting your results, explaining the reasons for observed differences, and discussing what your findings imply for real-world use cases.

4. Task Breakdown

This assignment consists of five tasks. Each task builds on the previous one, and together they form a complete data processing workflow from dataset selection through to reflection. Read each task description carefully before you begin work.

4.1 Task 1: Dataset Selection

What you need to do: Select a dataset that is larger than 700 megabytes from a reliable and publicly available source.

Where to find datasets: The most recommended sources are Kaggle (kaggle.com/datasets) and the UCI Machine Learning Repository (archive.ics.uci.edu). Both platforms host a wide range of real-world datasets across diverse domains including finance, transportation, healthcare, and social media.

What makes a dataset suitable: A good dataset for this assignment should be large enough to genuinely challenge your tools (above 700 MB), rich enough to support meaningful exploratory analysis, and diverse enough in its column types to allow data type optimisation. Avoid datasets that are simply large but contain only a single type of information.

Once you have selected your dataset, you must document it clearly. Your report should state the dataset name, its source URL, its file size, the domain it belongs to, and the approximate number of records it contains. This information goes into your Markdown report.

Dataset Suggestion Examples

Air Flight Analysis: Airline on-time performance data containing millions of flight records with columns for departure times, arrival times, delays, and airline codes.

NYC Taxi Trip Records: A large dataset of taxi journeys in New York City with pickup and dropoff coordinates, fare amounts, and passenger counts.

Open Government Data: National statistics datasets such as census records, public health records, or infrastructure usage data from government portals.

E-Commerce Transactions: Large transaction logs from retail platforms, useful for time series analysis and customer segmentation studies.

4.2 Task 2: Load and Inspect Data

What you need to do: Load your dataset into your Python environment using a method that handles its size efficiently, and then perform a basic inspection of its contents.

This task might sound straightforward, but it is actually one of the most important steps in any data project. The way you load data has a direct impact on whether your subsequent processing will succeed or fail. Simply using pandas.read_csv() on a 700 MB file without any optimisation will

often result in excessive memory consumption, slow loading, and in some cases a complete kernel crash in Google Colab.

After loading your data, you must include a brief inspection section in your notebook. This inspection should report the shape of the dataset (number of rows and columns), the names and data types of all columns, a summary of missing values, and a preview of the first few rows. This step demonstrates that you understand your data before you begin manipulating it.

> **Practical Tip: Loading Large Files in Google Colab**
>
> Google Colab provides a limited amount of RAM (typically around 12 GB on the free tier). To avoid running out of memory, consider connecting Google Drive to your Colab session and reading the file directly from Drive. You can also use the chunksize parameter in pandas.read_csv() to load the file in portions rather than all at once.

4.3 Task 3: Big Data Handling Strategies

What you need to do: Apply all five of the following big data handling strategies to your dataset, with clear code, explanation, and output for each one.

Strategy 1: Load Less Data

What it means: Instead of reading the entire file into memory at once, you load only the specific columns or rows that your analysis actually requires.

Why it is important: When a dataset has dozens of columns but your analysis only needs five of them, loading all columns wastes memory and slows processing. By specifying which columns to load using the usecols parameter, you immediately reduce memory consumption and speed up the read operation.

When to use it: Use this strategy at the very beginning of any data loading step. It is your first line of defence against unnecessary memory usage and should be applied whenever you know in advance which portions of the data are relevant to your task.

Strategy 2: Chunking

What it means: Instead of loading the full dataset in a single operation, you read it in smaller, manageable portions called chunks, process each chunk, and then either combine the results or discard each chunk before loading the next.

Why it is important: Chunking allows you to process a file that is larger than your available RAM by ensuring that only a small portion of it exists in memory at any one time. Without chunking,

attempting to load a 2 GB file into a machine with 4 GB of RAM would consume half of all available memory just for the raw data, leaving almost nothing for actual computation.

When to use it: Use chunking when you need to perform aggregate operations, filter rows, or transform columns on a file that is too large to load in one go. It is especially useful when you are working with limited memory environments such as the free tier of Google Colab.

Strategy 3: Data Type Optimisation

What it means: When Pandas loads a CSV file, it assigns default data types to each column, and these defaults are often wasteful. For example, a column of integers might be stored as int64 when the values never exceed a few thousand, meaning they could fit comfortably in an int16. Similarly, a column of repeated string categories can be stored as a category type rather than a full string object, drastically reducing memory.

Why it is important: Optimising data types can reduce a dataset's memory footprint by 50 percent or more, depending on its structure. This means faster operations, the ability to load more data at once, and significantly lower risk of out-of-memory errors.

When to use it: Apply this strategy after your initial inspection, once you understand the range of values in each column. It should be done before any major processing begins so that all subsequent operations benefit from the reduced memory footprint.

Strategy 4: Sampling

What it means: Sampling involves selecting a representative subset of your full dataset for analysis. Rather than working with all 10 million rows, you might work with a random sample of 100,000 rows, which is large enough to be statistically representative but small enough to process instantly.

Why it is important: In data science and engineering, it is standard practice to develop and test your analysis pipeline on a sample before applying it to the full dataset. Sampling dramatically shortens development cycles, allows for rapid iteration, and makes exploratory analysis feasible even on very large datasets.

When to use it: Use sampling during the early stages of your analysis when you are exploring data, testing code, and developing your processing logic. Once your code is validated on the sample, you can then apply it to the full dataset or to chunks of it. Note that sampling should not replace full processing in your final analysis; rather, it should complement it.

Strategy 5: Parallel Processing with Scalable Libraries

What it means: Parallel processing involves distributing a computational task across multiple processor cores simultaneously, so that different parts of the work are completed at the same time rather than sequentially. Libraries like Dask, PyArrow, and Polars are built to exploit this capability automatically.

Why it is important: Modern computers have multiple CPU cores, but standard Pandas operations are single-threaded, meaning they use only one core at a time. This leaves most of your computer's processing power idle. Scalable libraries take advantage of all available cores, which can reduce processing time by a factor proportional to the number of cores available.

When to use it: Use parallel processing for large scale aggregation, filtering, joining, and reading operations. It is most beneficial when the dataset is large enough that single-threaded processing creates a noticeable bottleneck. This is the cornerstone of why scalable libraries exist, and it is the primary reason why organisations move beyond Pandas for production data pipelines.

5. Comparative Analysis

The comparative analysis section is one of the most heavily weighted parts of this assignment, and it is also one of the areas where students most commonly lose marks. Please read this section carefully.

5.1 What You Must Compare

You are required to compare your **Pandas baseline implementation** against each of your **two scalable libraries**. This means you need to run the same or equivalent operations using all three libraries and record the results for each. Your comparison should cover the following three metrics.

- Memory Usage: Measure how much RAM each library consumes when loading and processing the dataset. You can use Python's memory_profiler library or the tracemalloc module to capture this information. Record peak memory usage in megabytes.
  - Use tools such as tracemalloc or memory_profiler.
  - Record before and after memory snapshots.
  - Report peak memory consumption in megabytes.

- Execution Time: Measure how long each library takes to complete the same operations. You can use Python's time module or the timeit module to record this. Report times in seconds.
  - Use time.time() or the %timeit magic command in Jupyter notebooks.
  - Measure loading time, processing time, and total time separately.
  - Run each operation multiple times and report the average.

- Processing Efficiency: Discuss qualitatively how straightforward it was to implement each approach, how well each library handles the full dataset without errors, and whether any library required special configuration or workarounds.
  - Note any limitations or errors encountered.
  - Comment on ease of implementation.
  - Discuss scalability potential for even larger datasets.

5.2 How to Present Your Findings

Clear, well-structured presentation of your results is essential. Markers should be able to look at your comparative analysis and immediately understand what you found without having to hunt through dense paragraphs of text. Use the following presentation formats.

Comparison Tables: Create a summary table that shows memory usage and execution time side by side for all three libraries. A clear table makes it easy to compare values at a glance and demonstrates that you have conducted a systematic analysis.

Charts and Graphs: Where possible, supplement your tables with bar charts or line graphs. A bar chart comparing execution times across the three libraries is much more visually compelling than a paragraph describing the numbers. Use matplotlib or seaborn to generate these charts directly in your notebook.

Critical Discussion: Numbers alone do not earn high marks. You must interpret what your results mean. If Polars is significantly faster than Pandas, why is that the case? What architectural or design features of Polars explain the difference? What are the trade-offs? This analytical depth is what separates Pass-level work from Distinction and High Distinction work.

6. Required Submission Table

Every group must include a submission table in the course GitHub repository. This table registers your group's library choices, dataset, and GitHub link in a standardised format. It allows the teaching team to quickly verify your submission and locate your work.

6.1 Table Format and Rules

The table must contain the following five columns, filled in according to the rules described below.

Team	Library 1	Library 2	Library 3	Dataset	Open in GitHub
Your Group Name	**Pandas (Compulsory)**	e.g. Dask	e.g. Polars	Dataset name and source	GitHub Badge Link
Sample1	**Pandas**	Dask	Koalas	Air Flight Analysis	GitHub Link

The following rules govern how you complete this table.

- Library 1 must always be Pandas. This column is not a free choice. Every group in the class uses Pandas as Library 1 because it is the baseline against which all comparisons are made.
- Library 2 and Library 3 must both be scalable libraries. You may choose from Dask, PyArrow, or Polars. Both choices must be different from each other. You may not use two of the same library.
- The Dataset column must clearly state the name of your dataset and ideally include a brief indication of its source, such as Kaggle or UCI.
- The GitHub link must be a working URL that directs the marker directly to your group folder within the bdm/ directory of the course repository. A broken link will result in a deduction for the GitHub Submission criteria.

7. Conclusion and Reflection

Task 5 of this assignment asks you to write a conclusion and reflection section in your Markdown report. This section is worth 10 marks and is your opportunity to demonstrate the quality of your learning and your ability to think critically about what you have done.

7.1 What to Summarise

Your conclusion should briefly summarise the key observations from your comparative analysis. Do not simply repeat the numbers from your results tables. Instead, identify the most significant findings. For example, you might note which library was most efficient for loading large files, which strategy had the greatest impact on memory consumption, or which approach you would choose for a production pipeline and why.

7.2 How to Reflect on Your Learning

Reflection is different from summarisation. While summarisation reports what happened, reflection asks you to consider what it means for your development as a data practitioner. Think about what surprised you during this assignment. Were there any strategies that performed differently from what you expected? Did you encounter any challenges that changed your understanding of how large-scale data processing works? What would you do differently if you were to repeat this assignment?

7.3 The Importance of Scalability

Your reflection should include a discussion of scalability. Think beyond the 700 MB dataset you used in this assignment. What would happen if the dataset were 10 GB, 100 GB, or 1 TB? Which of the strategies and libraries you used would still be viable at that scale? Which would require further upgrades to distributed systems like Apache Spark or cloud-based solutions? Showing that you can think ahead to the limits of the tools you have used demonstrates a level of understanding that is characteristic of High Distinction work.

8. Submission Requirements

Please read this section carefully before you submit. Incorrectly structured submissions or missing files will result in mark deductions, and late submissions will not be entertained under any circumstances.

8.1 GitHub Folder Structure

Your submission must follow a specific folder structure within the course GitHub repository. All files must be placed inside your group folder within the bdm/ directory as shown below.

Required Folder Structure

ass2/your_group/

big_data.md	**(Main Markdown report)**
readme.md	**(Brief introduction and links)**
big_data.ipynb	**(Colab Notebook with code)**

8.2 File Descriptions

big_data.md: This is your main written report. It should be a well-structured Markdown document that covers all tasks, contains your comparative analysis, and concludes with your reflection. It must include explanations, code snippets formatted in Markdown code blocks, and any tables or charts you have created.

readme.md: This is a brief introductory document at the group folder level. It should identify your group members, state the dataset you used, and provide a link to the main report and notebook.

big_data.ipynb: This is your annotated Jupyter notebook containing all of your working Python code. The notebook must be fully executed with all outputs visible. Code cells must be accompanied by Markdown cells that explain what the code does and why. The notebook should be runnable on Google Colab with minimal setup.

8.3 Group Rules

This assignment is a group project. Each group must consist of exactly two students. Both group members are expected to contribute to the work, and both will receive the same mark unless there is compelling evidence of highly unequal contribution that is brought to the attention of the lecturer before the submission deadline.

Academic integrity applies fully to this assignment. All code must be your own work or properly attributed to its source. Copying code from other groups or from online sources without attribution constitutes plagiarism and will result in a mark of zero and referral to the disciplinary committee.

9. Evaluation Rubric

Your assignment will be assessed against the following rubric. Understanding how marks are distributed will help you allocate your time appropriately and ensure that you focus on the areas that carry the most weight.

Assessment Criteria	Marks	What Markers Look For
Dataset Description	**5**	Clear source, size, domain, and record count.
Strategy Implementations	**30**	All 5 strategies applied with code, output, and explanation.
Comparative Analysis	**15**	Metrics recorded, tables or charts used, critical discussion present.
Code Functionality (Notebook)	**25**	Code runs without errors, outputs visible, well-annotated.
Markdown Clarity and Documentation	**10**	Clear writing, proper formatting, code blocks, no major grammatical errors.
Conclusion and Reflection	**10**	Thoughtful summary, genuine reflection, and scalability discussion.
GitHub Submission	**5**	Correct folder structure, all files present, working link in table.
TOTAL	**100**	

9.1 Tips for Achieving High Marks

The strategy implementations section carries 30 marks, which is the largest single allocation. Ensure that every strategy has three components: a code implementation that actually runs, visible output from that code, and a written explanation of what the code does and why it is useful.

The comparative analysis section at 15 marks rewards depth. Do not just record numbers. Explain them. A submission that says 'Polars was 3.2 times faster than Pandas because it uses lazy evaluation and vectorised operations written in Rust' will score significantly higher than one that simply states 'Polars was faster.'

Code functionality carries 25 marks and is heavily influenced by whether your notebook can be run from top to bottom without errors. Always re-run your entire notebook before submitting to confirm that all cells execute cleanly with all outputs visible.

10. Practical Tips and Common Mistakes

10.1 Practical Tips

Using Google Colab:

- Connect your Google Drive to Colab at the start of every session using drive.mount('/content/drive'). Store your dataset in Drive so that it persists between sessions.
- If your session disconnects, all local (non-Drive) data is lost. Always save intermediate results to Drive or to your GitHub repository.
- Use the Colab Pro tier if you find that you are consistently running out of memory. The free tier is sufficient for most datasets, but 700 MB to 1 GB datasets can sometimes be tight.
- Use the !pip install command in Colab to install any library not already available, such as polars or memory_profiler.

Handling Large Datasets:

- Always run your code on a small sample first before applying it to the full dataset. This saves significant time during development and debugging.
- Use the dtype parameter in pandas.read_csv() to specify column types at load time, rather than converting them after the fact. This is more efficient and reduces the peak memory spike that occurs during loading.
- If you are using Dask, remember that Dask operations are lazy by default. You must call .compute() at the end to trigger actual execution. Without it, you will only have a task graph, not a result.
- Monitor your memory usage as you work. In Colab, you can see your RAM usage in the top right corner of the interface. If it approaches the limit, restart the runtime and clear variables you no longer need.

Writing Clear Markdown:

- Use heading levels consistently. Use # for top-level sections, ## for subsections, and ### for sub-subsections.
- Always wrap code in triple backtick code fences with the language identifier, for example python at the start.
- Use tables and bullet points to present structured information. Markdown tables are straightforward to create and greatly improve readability.
- Proofread your Markdown file carefully. Spelling errors and unclear sentences reduce the perceived quality of your work even when the technical content is sound.

10.2 Common Mistakes to Avoid

Warning: These Mistakes Will Cost You Marks

Using a dataset that is too small. This is the most common mistake. If your dataset is below 700 MB, you are not meeting the minimum requirement of the assignment. Do not attempt to use a smaller dataset and justify it by saying the processing was still slow. The 700 MB threshold is a hard requirement.

Not including a comparison. Some students implement all five strategies but never actually compare Pandas to their scalable libraries with measured metrics. Without a structured comparison showing memory usage and execution time, you cannot score marks in the comparative analysis section, which is worth 15 points.

Weak or missing explanation. Code without explanation is not sufficient. Every code cell in your notebook must be accompanied by a Markdown explanation. Every strategy in your report must include a discussion of what it is, why it is used, and what the results show.

Submitting a notebook with cells that have not been run. Your notebook must be submitted in an executed state. All code cells must show their output. A notebook with empty output cells tells the marker that you have not actually verified that your code works.

Incorrect GitHub folder structure. Submitting files in the wrong location or with incorrect names means markers cannot locate your work. Follow the required folder structure exactly as described in Section 8.

11. Step-by-Step Workflow

The following workflow is recommended to help you plan and execute this assignment in an organised manner. Following this sequence will help ensure that you do not miss any critical steps.

Step	Phase	Actions
1	**Dataset Selection**	Browse Kaggle or UCI for datasets above 700 MB. Record the dataset name, source, size, domain, and record count. Download the dataset and upload it to Google Drive.
2	**Environment Setup**	Open Google Colab. Mount Google Drive. Install required libraries using !pip install for any that are not pre-installed. Import all necessary libraries at the top of your notebook.
3	**Initial Data Loading**	Load the dataset using Pandas with basic settings. Perform an inspection: check shape, column names, data types, missing values, and preview rows. Document all findings.
4	**Apply Strategy 1**	Reload the dataset using only the required columns. Document memory usage before and after. Explain what you did and why.
5	**Apply Strategy 2**	Implement chunked loading. Process each chunk to demonstrate the technique. Measure and record execution time.
6	**Apply Strategy 3**	Identify columns that can be downcasted. Apply type conversion. Measure memory reduction. Document the reduction percentage.
7	**Apply Strategy 4**	Apply random or stratified sampling. Use the sample for a quick exploratory operation. Document the trade-offs.
8	**Apply Strategy 5**	Implement the same operations using your two scalable libraries. Measure memory and time for each. Ensure code is clean and annotated.
9	**Comparative Analysis**	Compile all measurement results into comparison tables. Generate charts. Write your critical analysis interpreting what the results mean.
10	**Write Report**	Complete big_data.md with all sections. Ensure all code snippets are included as formatted code blocks. Proofread carefully.
11	**Final Check**	Re-run the entire notebook from top to bottom. Fix any errors. Verify all outputs are visible. Verify all files are in the correct GitHub folder.
12	**Submit**	Push all files to GitHub. Complete the submission table with your library choices, dataset, and GitHub link. Confirm the link works.

12. Submission Checklist

Before you push your final submission to GitHub, go through this checklist item by item. If you cannot check off every item on this list, your submission is not yet complete.

12.1 Dataset

- ☐ Dataset is sourced from a reputable platform such as Kaggle or UCI.
- ☐ Dataset size is confirmed to be larger than 700 MB.
- ☐ Dataset name, source, size, domain, and record count are documented in your report.

12.2 Strategies

- ☐ Load Less Data strategy is implemented with code and explanation.
- ☐ Chunking strategy is implemented with code and explanation.
- ☐ Data Type Optimisation strategy is implemented with code and explanation.
- ☐ Sampling strategy is implemented with code and explanation.
- ☐ Parallel Processing strategy is implemented using two scalable libraries.
- ☐ Each strategy includes output or screenshots showing it worked.

12.3 Libraries

- ☐ Library 1 is confirmed as Pandas.
- ☐ Library 2 is one of: Dask, PyArrow, or Polars.
- ☐ Library 3 is one of: Dask, PyArrow, or Polars (and is different from Library 2).
- ☐ All three libraries are used in your code.

12.4 Comparative Analysis

- ☐ Memory usage is measured and recorded for all three libraries.
- ☐ Execution time is measured and recorded for all three libraries.
- ☐ A comparison table is included in your report.
- ☐ At least one chart is included to visualise the comparison.
- ☐ A written discussion interprets the results critically.

12.5 Files and Submission

- ☐ big_data.md is complete and well-formatted.
- ☐ readme.md is present and contains group member names and links.
- ☐ big_data.ipynb has been fully re-run with all outputs visible.
- ☐ All files are in the correct bdm/your_group/ folder in GitHub.
- ☐ The submission table is completed with correct library names, dataset, and GitHub link.
- ☐ The GitHub link in the submission table has been tested and is working.
- ☐ No late submission. Deadline is 4 May 2025 at 5:00 PM.

13. Sample Structure for big_data.md

The following is a recommended structure for your main Markdown report file. This is a guide, not a template to be copied word for word. Your report must contain your own analysis, findings, and reflections based on your actual work.

Recommended big_data.md Structure

 # Assignment 2: Mastering Big Data Handling

Group Information

State your group name and the names of both members.

1. Dataset Description

Dataset name, source URL, file size, domain, number of records, and a brief description of the data.

2. Library Choices

State your three libraries: Pandas (Library 1), your chosen Library 2, and your chosen Library 3. Briefly justify why you selected Library 2 and Library 3.

3. Data Loading and Inspection

Show your initial loading code and the results of your data inspection.

4. Big Data Handling Strategies

4.1 Load Less Data | ### 4.2 Chunking | ### 4.3 Data Type Optimisation | ### 4.4 Sampling | ### 4.5 Parallel Processing

For each strategy: explanation, code snippet, output or screenshot, and discussion.

5. Comparative Analysis

Memory usage table, execution time table, comparison chart, and critical discussion of findings.

6. Conclusion and Reflection

Summary of key observations, personal reflection on learning, and discussion of scalability.

References

Citation of dataset source and any libraries or resources consulted.